```
makeDbOperator = (db) ->
  (operation) ->
    (entry, params...) ->
      db[operation] entry, params...
```

You only want to cache loading of data, not saving of data, and the `makeDbOperator` abstraction doesn't know anything about loading and saving. This is a clear indication that `makeDbOperator` is the wrong place to implement this caching; if it has to know the difference between a load and a save, it has to know too much. The correct place to implement the caching is where you define `loadProductData`:

```
productDataCache = Object.create null
loadProductData = (name, callback) ->
  cachedCall = (makeDbOperator products) 'get'
  if productDataCache.hasOwnProperty name
    console.log 'cache hit'
    console.log productDataCache[name]...
    callback productDataCache[name]...
  else
    cachedCall name, (results...) ->
      productDataCache[name] = results
```

Remember from the previous chapter that passing null to Object.create means that the created object has null as a prototype and so doesn't inherit any properties you don't want.

What if you need to cache something else in the future? You need a function that can cache *any* function.

General cache

As with previous examples, you can extract such a general function from the specific solution you already have. Note that for simplicity this solution is intended to work by using the first argument as the key and the last argument as the callback:

```
withCachedCallback = (fn) ->
  cache = Object.create null
  (params...) ->
    key = params[0]
    callback = params[params.length - 1]
    if key of cache
      callback cache[key]...
    else
      paramsCopy = params[..]
      paramsCopy[params.length-1] = (params...) ->
        cache[key] = params
```